CS-104 Quiz 3 (ANSWER KEY)

Fall 2026

Name: _____

This is an in-class, closed-book quiz (30 points). If you have any questions, please feel free to ask them. When you are finished, you can turn it in and go.

1. (4 points)

Write a function called `trapezoid_area` that receives the lengths of the two parallel sides (`base1` and `base2`) and the height (`height`) of a trapezoid, and returns its area. This can be computed as:

area = average_base × height

where `average_base` is the average (mean) of `base1` and `base2`. Be sure to use appropriate Python syntax. (You do not need to provide an algorithm, documentation, or test cases.)

```
def trapezoid_area(base1, base2, height):
    average_base = (base1 + base2) / 2
    return average_base * height
```

Grading: header with three parameters 1, average 1, area formula 1, `return` (not `print`) 1.

2. (3 points)

Using your `trapezoid_area` function from the previous question, write a code segment that computes the **total** area of two trapezoids, one with bases 3 and 5 and height 2, and one with bases 4 and 6 and height 3, and prints the total rounded to 1 decimal place.

```
total = trapezoid_area(3, 5, 2) + trapezoid_area(4, 6, 3)
print(round(total, 1))
```

Prints `23.0`. Grading: two correct calls 2, rounding and printing 1.

3. (4 points)

What does this code print? Write every line of output.

```
def mystery(a, b):
    print("start", a)
    if a > b:
        return a - b
    return b * 2

# --- main program ---
x = mystery(5, 2)
y = mystery(1, 4)
z = mystery(b=3, a=3)
print(x, y, z)
```

`start 5`, `start 1`, `start 3`, `3 8 6`. **Grading: 1 point per line.**

4. (3 points)

The code below crashes. What is printed before the crash, what error happens and why, and how would you fix the function?

```
def add_tax(price):
    total = price * 1.06
    print(total)

# --- main program ---
cost = add_tax(100) + 5
print(cost)
```

It prints `106.0`, then raises a `TypeError`: the function has no `return`, so `add_tax(100)` gives `None`, and `None + 5` fails. Fix: replace `print(total)` with `return total`. Grading: 1 point each.

5. (6 points)

Write a function called `count_in_range` that receives a list of numbers `values` and two numbers `low` and `high`, and returns how many elements of `values` are between `low` and `high` (inclusive). For example, `count_in_range([3, 8, 1, 9, 5], 3, 8)` returns `3`. Then write one line that calls your function with that same example and prints the result.

```
def count_in_range(values, low, high):
    count = 0
    for v in values:
        if low <= v <= high:
            count = count + 1
    return count

print(count_in_range([3, 8, 1, 9, 5], 3, 8))
```

Grading: header 1, counter and loop 2, inclusive condition 1, `return` after the loop 1, call and print 1.

6. (3 points)

What does each `print` statement output (1 point each)? In one sentence, explain why the first result is so different from the second.

```
import numpy as np
values = [1, 2, 3]
arr = np.array(values)
print(values * 2)
print(arr * 2)
print(arr ** 2)
```

`[1, 2, 3, 1, 2, 3]`, `[2 4 6]`, `[1 4 9]`. Multiplying a list by an integer repeats the list, while arithmetic on a NumPy array is applied to each element. Grading: 2 points for the first two lines with the explanation, 1 point for the third.

7. (3 points)

Give the array each expression creates (1 point each). Assume `import numpy as np`.

```
>>> np.arange(3)
```
[0 1 2] — example answer

```
>>> np.arange(0, 10, 3)
```

```
>>> np.linspace(0, 1, 3)
```

```
>>> np.arange(1, 2, 0.25)
```

`[0 3 6 9]`, `[0. 0.5 1.]`, `[1. 1.25 1.5 1.75]`.

8. (4 points)

When a capacitor charges in an RC circuit, its voltage after `t` seconds is

v = 5 × (1 - e^(-t / 2))

Write a code segment that uses NumPy to create 200 evenly spaced times from 0 to 10 seconds, computes the voltage at each time (use `np.exp`), and uses Matplotlib to plot voltage versus time with the axis labels `Time (s)` and `Voltage (V)` and a title.

```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 10, 200)
v = 5 * (1 - np.exp(-t / 2))
plt.plot(t, v)
plt.xlabel("Time (s)")
plt.ylabel("Voltage (V)")
plt.title("Charging Capacitor")
plt.show()
```

Grading: `linspace` 1, voltage formula with `np.exp` on the array 2, plot with labels, title, and `show` 1.